

Static Assets Differences with PWA Kit

PWA Kit uses a traditional Webpack-based approach with SVG sprites and utility functions for responsive images. Storefront Next leverages Vite's native asset handling with a modern component-based approach for responsive images and tree-shaken icon imports. The shift simplifies asset management while providing more powerful responsive image capabilities through the `DynamicImage` component.

Contents

Key Differences

- Image Import Patterns
- Icon Usage
- Responsive Image Handling
- Key Similarities
- Storefront Conversion Considerations

Key Differences

Aspect	PWA Kit	Storefront Next
Build Tool	Webpack	Vite
Static Directory	<code>app/static/</code>	<code>public/</code>
Asset URLs	<code>getAssetUrl()</code> helper	Direct <code>/</code> paths
Icon System	SVG sprite sheet	Tree-shaken React icons
Image Component	Custom with SSR preload	<code><picture></code> with React 19 preload
Responsive Images	Utility functions	<code>DynamicImage</code> component
Font Strategy	Chakra UI defaults	Tailwind CSS defaults
CSP Headers	Explicit in <code>ssr.js</code>	Framework-handled

Image Import Patterns

PWA Kit:

```
1 import { getAssetUrl } from "@salesforce/pwa-kit-react-renderer"
2
```

[Try for free](#)

```
1 import heroImage from "/images/hero.png";
2 // or
3 const src = "/images/hero.png";
```

Icon Usage

PWA Kit:



```
1 // Import registers to sprite
2 import "@salesforce/retail-react-app/app/assets/sv
3
4 // Use via sprite reference
5 <Icon>
6   <use xlinkHref="#search" />
7 </Icon>;
```

Storefront Next:



```
1 import { Search } from "lucide-react";
2
3 <Search className="h-5 w-5" />;
```

Responsive Image Handling

PWA Kit:



```
1 // Manual utility function usage
2 const {srcSet, sizes} = getResponsivePictureAttrib
3   src: imageUrl,
4   breakpoints: [320, 640, 1024]
5 })
6
7 <picture>
8   <source srcSet={srcSet} sizes={sizes} />
9   <img src={imageUrl} alt={alt} />
10 </picture>
```

Storefront Next:



```
1 // Component handles complexity
2 <DynamicImage
3   src={imageUrl}
4   alt={alt}
5   widths={{ base: "100vw", md: "50vw", lg: "33vw"
6   priority="high"
7 />
```

[Try for free](#)

Service (`?sw={width}`).

3. System fonts: Neither bundles custom web fonts.
4. SSR awareness: Both handle preloading for above-fold images.
5. Aspect ratio handling: Both manage layout stability.

Storefront Conversion Considerations

- Asset Path Changes
 - Replace `getAssetUrl()` with direct `/` paths.
 - Move files from `app/static/` to `public/`.
- SVG Icons
 - Replace sprite imports with `lucide-react` icons.
 - Or create individual SVG component imports.
- Image Component
 - Replace custom Image with `DynamicImage`.
 - Update width props to new format.
- Responsive Images
 - Convert utility function usage to `DynamicImage` props.
 - Breakpoint format changes from array to object.

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Community
Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects



Try for free

